Section A: Concept Application

(Theoretical - Scenario Based)

1. Food Delivery Application: Compiler vs. Interpreter

- **Preferred Stage for Compiler:** You would prefer a compiler during the **final deployment and production stage**.
- **Why? (Based on the criteria):**
    - **Execution Speed:** Compilers translate the entire code into machine language beforehand, resulting in significantly faster execution speeds. For a food delivery app handling real-time GPS tracking and simultaneous orders, high performance is critical.
    - **Error Detection:** Compilers detect all syntax errors at once during compilation, ensuring that the app won't crash mid-operation due to basic syntax flaws.
    - **Deployment:** A compiled application is packaged into a standalone executable, meaning you do not need to ship the source code or an interpreter to the end-users, protecting your intellectual property and optimizing resource usage. (Note: An interpreter is preferred during the initial development/debugging phase for quick, line-by-line testing.)

2. Employee Salary Calculation Constructs

To efficiently design a program calculating salary with multiple conditions (bonus, tax, overtime), you should combine the following programming constructs:

- **Sequential Structure:** For standard, linear operations like inputting basic employee details and calculating basic pay.
- **Conditional Statements (if-else ladder):** To apply different tax brackets (e.g., if salary > 50,000, tax = 20%; else if salary > 30,000, tax = 10%).
- **Relational and Logical Operators (&&, ||):** To evaluate complex conditions, such as checking if an employee qualifies for a bonus based on both performance ratings *and* attendance.

3. Menu-Driven Console Applications: Loops vs. Conditional Repetition

- **Why Loops are Preferred:** Conditional repetition (like using if statements with goto labels) makes code unreadable, disorganized ("spaghetti code"), and difficult to debug. Loops provide a structured, clean, and controlled environment for repeating actions until a specific exit condition is met.
- **Loop Choice:** A **do-while loop** is the ideal choice for a menu-driven program.

- **Why?** A menu-driven application must display the options to the user **at least once** before checking if they want to exit. Because a do-while loop is an *exit-controlled loop*, it executes the body first and evaluates the condition at the end, perfectly matching this logic.

4. Average Marks Calculation: Data Type Issue

- **Probable Cause:** The student is experiencing **Integer Division**. In programming languages like C/C++, dividing an int by an int results in an int, discarding any fractional parts (e.g., $15 / 2$ becomes 7 instead of 7.5).
- **Solution:** Change the variable storing the total marks or the average to a floating-point data type (**float** or **double**). Alternatively, use explicit type casting during division:
- C

average = (float)total_marks / total_subjects;

-
- This preserves the decimal values and ensures accurate results.

5. Runtime Crash Debugging

- **Category of Error:** This is a **Runtime Error** (specifically, exceptions like Segmentation Fault, Division by Zero, or Null Pointer Dereference).
- **Debugging Practices to Resolve It:**
    1. **Print Statement Debugging (Traces):** Insert temporary print statements (printf or cout) right before critical blocks to isolate exactly which line of code causes the crash.
    2. **Using a Debugger (GDB / IDE Debugger):** Run the program through a debugger to inspect variables, set breakpoints, and view the **call stack** at the exact moment the crash occurs.

6. Structures vs. Multiple Arrays for Student Records

- **Why Choose a Structure (struct):** Multiple parallel arrays (e.g., char names[100][50], int marks[100], char grades[100]) keep related data disconnected, making sorting, deleting, or passing records to functions highly error-prone.
- A **Structure** acts as a single entity that binds heterogeneous data types (name, marks, grade) together into a single logical record. This mimics real-world objects, reduces code complexity, and improves data integrity and readability.